

Budapesti Műszaki és Gazdaságtudományi Egyetem

Villamosmérnöki és Informatikai Kar

Automatizálási és Alkalmazott Informatikai Tanszék

LLM-ALAPÚ HR ASSZISZTENS

Önéletrajz anonimizálás és rangsorolás nagy nyelvi modellek segítségével

Önálló Laboratórium Dokumentáció

Készítette:

Vaska Barnabás

H7RJ0U

Konzulens:

Forstner Bertalan

Budapest, 2026.05.25.

Összefoglaló

A dokumentáció egy LLM-alapú HR asszisztens rendszer tervezéséről és megvalósításáról számol be, amelyet az önálló laboratóriumi munka keretében, tizennégy héten keresztül fejlesztettem. A rendszer az előítélet mentes (blind recruitment) toborzási folyamatot támogatja. Célja, hogy a HR szakember az önéletrajzokat kizárólag szakmai tartalmuk alapján értékelhesse, személyes azonosításra alkalmas adatok és a nemre utaló nyelvi fordulatok nélkül.

Az anonimizálás öt specializált LLM-agent segítségével valósul meg, amelyek egymás után dolgozzák fel az önéletrajzot. Eltávolítják a személyes neveket, semlegesítik a nemi töltetet hordozó munkakör megnevezéseket és névmásokat, illetve maszkolják a családi állapotra és nemhez köthető életesemény utalásokra vonatkozó részeket. A folyamat önellenőrző validációs lépéssel egészül ki, amely garantálja, hogy az anonimizálás ténylegesen megtörtént, és szükség esetén automatikusan újraindítja a feldolgozást. Az anonimizált szövegekből az LLM strukturált jelölt-profilt állít elő, amelyet a rendszer vektoros adatbázisban tárol, majd a pozíció leírásával összevetve rangsorolja a jelölteket és kritériumalapú indoklást generál.

Az elkészült webalkalmazás két szerepkört kezel. A HR-szerepkörű felhasználó álláshirdetéseket hozhat létre, konfigurálhatja az anonimizálást, megtekintheti az anonimizált önéletrajzokat, és LLM-alapú rangsorolást indíthat a jelöltekről. A Jelentkező-szerepkörű felhasználó a nyitott állások listájából választhat, és PDF önéletrajzot tölthet fel. A rendszer Python alapú backendjét FastAPI, LangChain és LangGraph keretrendszerekkel, frontendét Next.js-sel valósítottam meg. A helyességet tizenöt egységtesztrel, a RAG-rendszer minőségét RAGAS metrikákkal igazoltam.

1. Bevezetés

A toborzási folyamat egyik legkritikusabb, mégis legkevésbé szabályozott lépése az önéletrajzok előszűrése. A HR szakemberekre háruló adminisztrációs teher miatt egy átlagos álláshirdetésre beérkező több száz pályázat mindegyike csupán néhány másodpercnyi figyelmet kap. Ezt a problémát tovább súlyosbítja az emberi döntéshozatalban akaratlanul is jelenlévő tudattalan torzítás (unconscious bias), amely bizonyos demográfiai csoportokat, elsősorban nőket és kisebbségeket, hátrányosan érinthet még a tartalmi értékelés előtt, pusztán a jelölt nevéből, fényképéből vagy a szövegben fellelhető nemi utalásokból kiindulva.

A jelenleg elterjedt kulcsszó alapú szűrőszoftverek sem nyújtanak érdemi megoldást, hiszen merevek és kontextusérzéketlenek, és értékes jelölteket szűrhetnek ki pusztán amiatt, hogy az önéletrajzban más szinonimát használtak a keresett készségre, miközben a tudattalan torzítás problémáját egyáltalán nem kezelik.

Az önálló laboratóriumi munka célja egy olyan fejlett dokumentumfeldolgozó és döntéstámogató rendszer megtervezése és implementálása, amely képes az önéletrajzok automatikus anonimizálására és strukturált elemzésére. A feladat négy egymásra épülő modult foglalt magában. Egy dokumentumfeldolgozó pipeline megvalósítását, egy LLM-alapú anonimizáló ágens fejlesztését, egy RAG-technológián alapuló rangsoroló rendszer felépítését, valamint egy webes felülettel rendelkező kliens-szerver alkalmazás elkészítését. A félév végére a rendszer képes PDF önéletrajzokat fogadni, anonimizálni, elemezni, majd az álláspozíció leírásával összevetve rangsorolni a jelölteket részletes indoklással.

A fejlesztés módszertanát a fokozatos, iteratív megközelítés jellemezte. Egyes komponenseket az adott technológia tanulási fázisában, előbb egy elkülönített kísérletezési környezetben próbáltam ki, ahol gyorsan lehetett tesztelni és iterálni anélkül, hogy az éles rendszerben bármi változtatás lenne. Csak a jónak ítélt megoldások kerültek át a tényleges backend kódbázisba, ahol már egységteszttekkel is ellenőriztem a helyes működést. Ez a workflow lehetővé tette, hogy például az anonimizáló agent promptokat több iteráción keresztül finomítsam, és a változtatások hatását izoláltan mérjem, mielőtt azok bekerültek volna a pipeline-ba.

2. Felhasznált technológiák

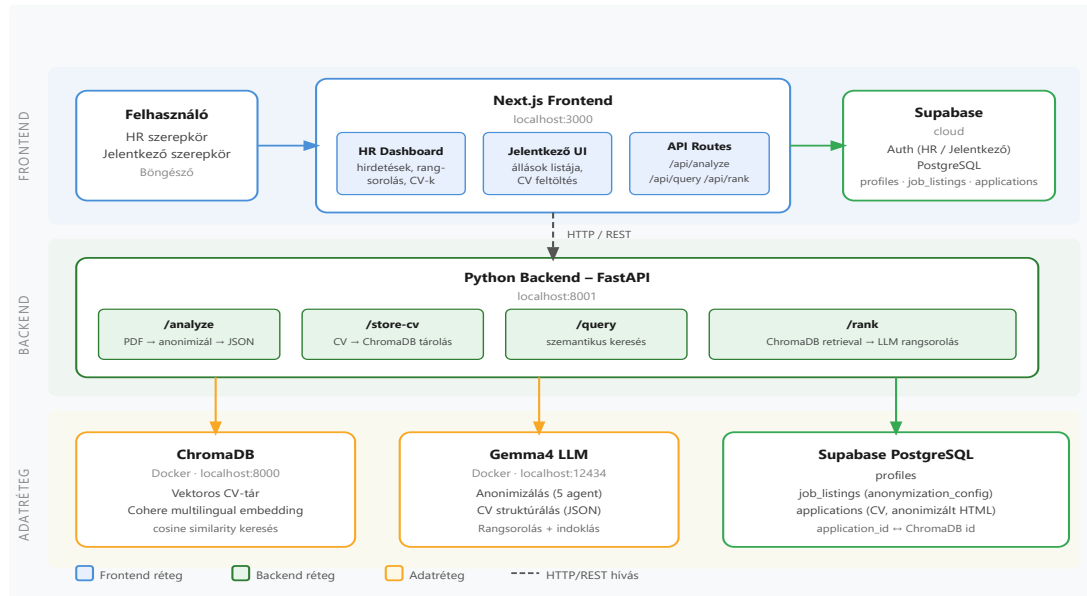
A fejlesztés során az eszközöket elsősorban az iparban széleskörűen elterjedt, aktív közösségi támogatással rendelkező megoldásokat kerestem. A cél az volt, hogy az alkalmazott technológiák ne csak a félév keretein belül működőképesek legyenek, hanem valós, éles környezetben is megállják a helyüket.

A rendszer backendje Python 3.12-n és FastAPI keretrendszeren alapul. A FastAPI-t azért választottam, mert az egyik legelterjedtebb Python-alapú REST API keretrendszer, amely aszinkron kiszolgálást, beépített Pydantic-alapú adatvalidációt és automatikus OpenAPI dokumentációgenerálást biztosít. Az LLM-hívások összeállítására LangChain-t és LangGraph-ot alkalmaztam. A LangChain az iparban szabvánnyá vált az LLM-alapú alkalmazások fejlesztésében. Egységes interfészt biztosít különböző modellszolgáltatókhoz, és megkönnyíti a prompt-sablonok, a modellhívások és a kimeneti parserek összekapcsolását. A LangGraph ennek kiegészítéseként irányított gráf alapú, állapotteljes workflow-t tesz lehetővé, amelyen belül az egyes feldolgozási lépések jól elkülönített csúcspontokként jelennek meg, és a köztük átadott állapot típusbiztosan kezelhető.

A nagy nyelvi modell tekintetében a kísérletezési szakaszban a Google Gemini API-t használtam, mivel könnyen hozzáférhető és erős baseline teljesítményt nyújt. Az éles anonimizáláshoz és rangsoroláshoz azonban áttértem a lokálisan, Docker konténerben futtatott Gemma4 modellre. Ennek több oka is volt. A Gemini API napi korlátja a tesztelési iterációkat lassította, a lokális modell viszont offline, korlátlan hívásszámmal futtatható, és a fejlesztési ciklust érezhetően felgyorsítja. A Gemma4 a Google nyílt forráskódú model családjába tartozik, amely teljesítményben versenyképes az API-alapú alternatívákkal, miközben az adatok nem hagyják el a helyi környezetet, ami adatvédelmi szempontból is előnyös.

A PDF szövegkinyeréshez PyMuPDF könyvtárat alkalmaztam, amely karakterpontosságú szövegkiolvasást és fejlett PDF-kezelést biztosít. Az önéletrajzok vektoros tárolásához ChromaDB-t választottam, amelyet Docker konténerben futtattam helyi perzisztens volumennel, így az adatok az újraindítások között is megmaradnak. A szemantikus embeddinget a Cohere embed-multilingual-light-v3.0 modellje végzi, amely egységes vektortérbe képezi le a különböző nyelvű szövegeket, lehetővé téve a többnyelvű keresést. A relációs adatok tárolásához Supabase-t választottam, amely tartalmaz beépített PostgreSQL adatbázist, valamint hitelesítési rendszert, amit szintén felhasználtam az authentication-hoz. A frontend Next.js 14-en és TypeScripten alapul, amelyek server-side renderinget és típusbiztos API route-kezelést biztosítanak. A tesztelés során a pytest egységtesztelő keretrendszert és a RAGAS értékelési könyvtárat alkalmaztam.

3. Rendszerarchitektúra



1. ábra Rendszerarchitektúra blokkdiagram

A rendszer három jól elkülönített rétegből áll, amelyek HTTP REST API-n és adatbázis-kapcsolatokon keresztül kommunikálnak egymással. Ez a rétegelt felépítés biztosítja az egyes komponensek önálló fejleszthetőségét és tesztelhetőségét.

A frontend réteg egy Next.js alapú webalkalmazás, amely két különböző nézetet kínál a Supabase Auth által kezelt szerepkörök alapján. A HR-szerepkörű felhasználó álláshirdetéseket hozhat létre és kezelhet, megtekintheti a beérkezett, anonimizált önéletrajzokat, és LLM-alapú rangsorolást indíthat a jelöltek között. A Jelentkező-szerepkörű felhasználó a nyitott álláshirdetések listáját látja, és PDF önéletrajzot tölthet fel az adott pozícióra. A frontend Next.js API route-okon keresztül továbbítja a kéréseket a Python backendnek, amelyet a `BACKEND_URL` környezeti változó konfigurál.

A backend réteg egy FastAPI alkalmazás, amely négy REST végpontot kínál. Az `/analyze` végpont fogadja a feltöltött PDF fájlokat, lefuttatja az anonimizáló pipeline-t, kinyeri a strukturált jelölt-adatokat, és JSON válaszban visszaküldi az anonimizált HTML tartalmat. A `/store-cv` végpont a Supabase által kiosztott `application_id` azonosítóval tárolja a CV szövegét a ChromaDB vektoros adatbázisban. A `/query` végpont szöveges lekérdezés alapján szemantikus keresést végez a ChromaDB-ben. A `/rank` végpont a ChromaDB-ből visszakeresett, releváns önéletrajzokat az LLM segítségével rangsorolja az álláspozíció leírása szerint.

Az adatréteg két összetevőből áll. A ChromaDB vektoros adatbázis az önéletrajzok szemantikus reprezentációját tárolja, és cosine hasonlóság alapján képes a leginkább releváns CV-eket visszakeresni egy adott lekérdezéshez. A Supabase PostgreSQL adatbázis három fő táblát tartalmaz. A `profiles` tábla a felhasználói fiókokat kezeli, és a Supabase Auth rendszeréhez kapcsolódik, tárolja a felhasználó nevét és szerepkörét (hr vagy applicant). A `job_listings` tábla az álláshirdetések adatait tartalmazza. A pozíció nevét, leírását, nyitva/zárva állapotát, a létrehozás időpontját, valamint az `anonymization_config` JSON mezőt, amely azt rögzíti, hogy az adott hirdetésnél melyik anonimizáló agent van bekapcsolva. Az `applications` tábla a jelentkezéseket tárolja. A `job_listing_id` idegen kulcson keresztül kapcsolódik az

álláshirdetéshez, tartalmazza a jelölt strukturált adatait (név, e-mail, helyszín, skill-ek), az anonimizált önéletrajz HTML formátumban, a nyers CV-szöveget a rangsoroláshoz, az application_id-t (amely a ChromaDB-beli azonosítóval megegyezik), és a benyújtás időpontját.

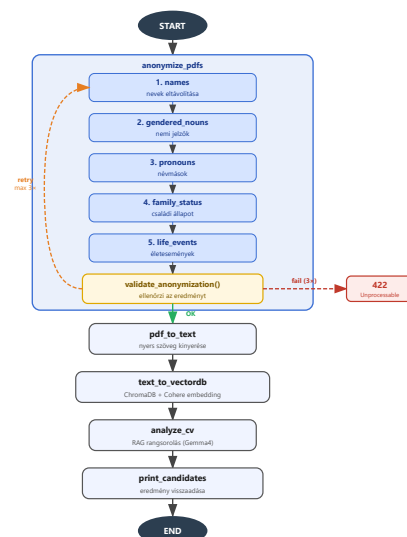
Egy tipikus jelentkezési folyamat a következőképpen zajlik. A Jelentkező feltölti a PDF önéletrajzát a frontend weboldalán, amely az /analyze végponton keresztül elküldi a Python backendnek. A backend lefuttatja az anonimizáló pipeline-t, kinyeri a strukturált adatokat, majd visszaküldi az eredményt. A frontend elmenti a jelölt adatait a Supabase-be, majd a /store-cv végponton keresztül a ChromaDB-be is betölti az önéletrajz szövegét. Amikor a HR szakember a rangsorolást kéri, a rendszer a /rank végponton keresztül lekérdezi a ChromaDB-t, majd az LLM elvégzi a rangsorolást és visszaküldi az eredményt.

4. Az önálló munka részletes bemutatása

4.1 Dokumentumfeldolgozó pipeline

Az önéletrajzok feldolgozása egy LangGraph StateGraph gráf mentén, egymást követő csúcspontokon keresztül valósul meg. A gráf állapota egy TypedDict struktúra, amely a feldolgozás minden fázisában tartalmazza az éppen rendelkezésre álló adatokat. A bemeneti PDF elérési útjait, az anonimizált szövegeket, a nyers szövegeket és a kinyert jelölt-adatokat. A LangGraph-alapú megközelítés azért bizonyult előnyösnek, mert az állapot explicit és típusbiztos, az egyes csúcspontok jól izoláltak és önállóan tesztelhetők, a feldolgozás hibái pedig pontos csúcspont-szinten lokalizálhatók.

Az első csúcspont (anonymize_pdfs) lefuttatja az anonimizáló pipeline-t minden bemeneti PDF-re. A második csúcspont (pdf_to_text) PyMuPDF segítségével kinyeri a nyers szöveges tartalmat az eredeti PDF-ekből, és ezt a szöveget tárolja el később a ChromaDB-ben a rangsoroláshoz. Fontos, hogy a ChromaDB-be nem az anonimizált, hanem az eredeti szöveg kerül. A rangsoroláshoz az eredeti, teljes tartalmú önéletrajz a releváns, mivel az anonimizált verzióból hiányoznak az egyébként a hasonlóság-mérés szempontjából hasznos tulajdonnevek és kontextusok. A harmadik csúcspont (text_to_vectordb) a kinyert szövegeket feltölti a ChromaDB-be a Cohere embeddingekkel, duplikátumszűréssel. A negyedik csúcspont (analyze_cv) az LLM-et hívja, amely a nyers szövegből strukturált JSON profilt állít elő, ami tartalmazza a nevet, e-mail-címet, telefonszámot, helyszínt, skill-eket és nyelveket.



2. ábra Langgraph állapotgép

4.2 Anonimizáló pipeline

Az anonimizálás az önálló munka fő területe volt. A pipeline öt, egymást követő LLM-agentet alkalmaz, amelyek mindegyike egyetlen, jól körülhatárolt feladatért felelős. Ezt a szétválasztást az indokolta, hogy ha egyetlen agent végezné el az összes anonimizálási feladatot, a feladatok közötti interferencia nehezen ellenőrizhető lenne mivel az agent hajlamos átfedő területeken is beavatkozni.

Az első agent (names) az önéletrajzban szereplő személyes neveket, névmódosítókat és megszólítási formákat azonosítja, és azokat a [CANDIDATE] helyőrzővel helyettesíti. A második agent (gendered_nouns) a nemi töltetet hordozó munkakör megnevezéseket semlegesíti. Például az angol stewardess kifejezést flight attendant-ra, a fireman szót firefighter-re, az actress szót actor-ra cseréli, magyar kontextusban pedig az üzletasszony-t üzleti szakemberré, a titkárnőt titkárrá alakítja. A harmadik agent (pronouns) a nemi névmásokat semlegesíti. A she és her névmásokat they, them, their alakra cseréli, a he, his, him névmásokat szintén they, them, their formákra, megőrizve a mondat eredeti szerkezetét. A negyedik agent (family_status) a családi állapotra és szerepekre vonatkozó kifejezéseket távolítja el vagy semlegesíti, mint például a married, husband, wife, children, mother, maiden name szavakat, amelyeket [PERSONAL DATA] helyőrzőre cserél. Az ötödik agent (life_events) a nemhez köthető életesemény-utalásokat kezeli. A maternity leave kifejezést career break-re, a paternity leave szintén career break-re, a military service szót public service-re alakítja.

Minden agent System-prompt alapján működik, amelyet gondosan hangoltam az agent határok pontos betartása érdekében. Például a pronouns agent kizárólag névmásokat cserélhet, de nem nyúlhat munkakörmegnevezésekhez, és a family_status agent a maternity leave kifejezéshez nem nyúlhat, azt a life_events agent kezeli. Ezt a különválasztást a tesztelés során felmerülő problémák indokolták. Az eredeti, általánosabb promptok esetén az agenthatárok átfedtek, és az egyik agent felülírta a másik munkáját.

4.3 Önellenőrző validációs mechanizmus

Mivel az LLM-alapú anonimizálás nem determinisztikus, az egyetlen átfutás nem garantálja, hogy minden tiltott mintát eltávolított a szövegből. Ennek kezelésére egy visszacsatolós validációs mechanizmust implementáltam, amely a feldolgozás részét képezi.

A pipeline feldolgozás előtt megvizsgálja az eredeti szövegben, hogy jelen vannak-e az egyes agentekhez tartozó ismert tiltott minták. Például a gendered_nouns agenthez stewardess, fireman, chairman, actress, a pronouns agenthez a she, he, his, her szavak, a family_status agenthez a married és a maiden name kifejezések, a life_events agenthez a maternity leave és a paternity leave szavak tartoznak. Csak azokat a mintákat ellenőrzi, amelyek ténylegesen jelen vannak az adott önéletrajzban, így elkerülhető, hogy olyan elemeket keressen, amelyek soha nem is szerepeltek a szövegben.

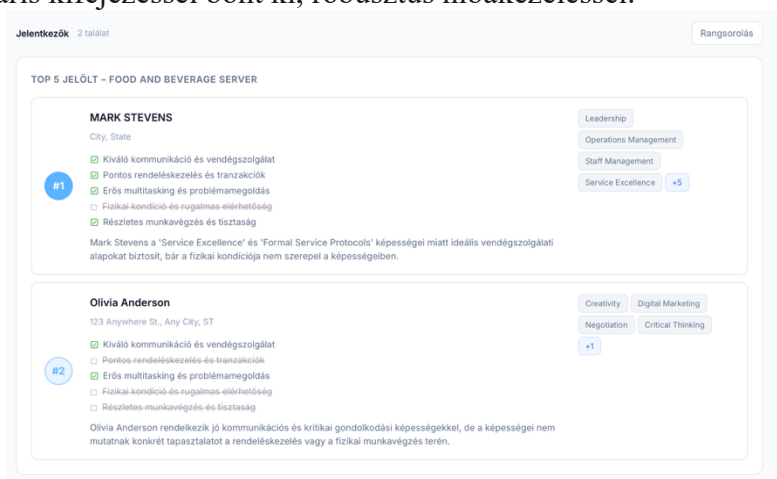
A feldolgozás után a rendszer ellenőrzi, hogy ezek a konkrét minták eltűntek-e az outputból. Ha a validáció megbukik, a pipeline automatikusan újraindul az eredeti PDF szövegéből, legfeljebb háromszor próbálkozva. Ha egyik kísérlet sem sikeres, a backend 422 Unprocessable Entity HTTP státusszal tér vissza, és a jelentkezés nem kerül mentésre. Ez a mechanizmus biztosítja, hogy csak ténylegesen anonimizált önéletrajzok kerüljenek az adatbázisba.

4.4 LLM-alapú rangsorolás kritériumértékeléssel

A rangsorolás a RAG (Retrieval-Augmented Generation) paradigmán alapul. Amikor a HR szakember rangsorolást kér egy adott álláshirdetésnél, a rendszer először a ChromaDB vektoros adatbázisból szemantikusan keresi vissza azokat a jelölteket, akiknek az önéletrajza el van tárolva és az álláspozíció leírásához a leghasonlóbbak. A cosine hasonlóság alapján rangsorolt találatok biztosítják, hogy az LLM-rangsoroló csak a releváns jelölt készlettel dolgozzon.

Az LLM rangsorolás kétlépéses folyamatot követ. Az első lépésben az LLM azonosítja a pozícióleírásból az öt legfontosabb követelményt, amelyek tömörek és konkrétan mérhetők. A második lépésben minden jelölnél megjelöli, hogy teljesíti-e ezeket a kritériumokat (igaz vagy hamis értékkel), és egy-két mondatos, konkrét magyar nyelvű indoklást generál, amely megnevezi, mit teljesít és mit nem az adott jelölt. Az eredmény a frontend rangsorolási kártyáin jelenik meg. Zöld pipa jelzi a teljesített, áthúzott szöveg a nem teljesített kritériumokat.

A rangsoroláshoz lokálisan futtatott Gemma4 modellt használtam. A prompt úgy van megfogalmazva, hogy az LLM kizárólag a megadott JSON struktúrában válaszoljon, amelyet a backend reguláris kifejezéssel bont ki, robusztus hibakezeléssel.



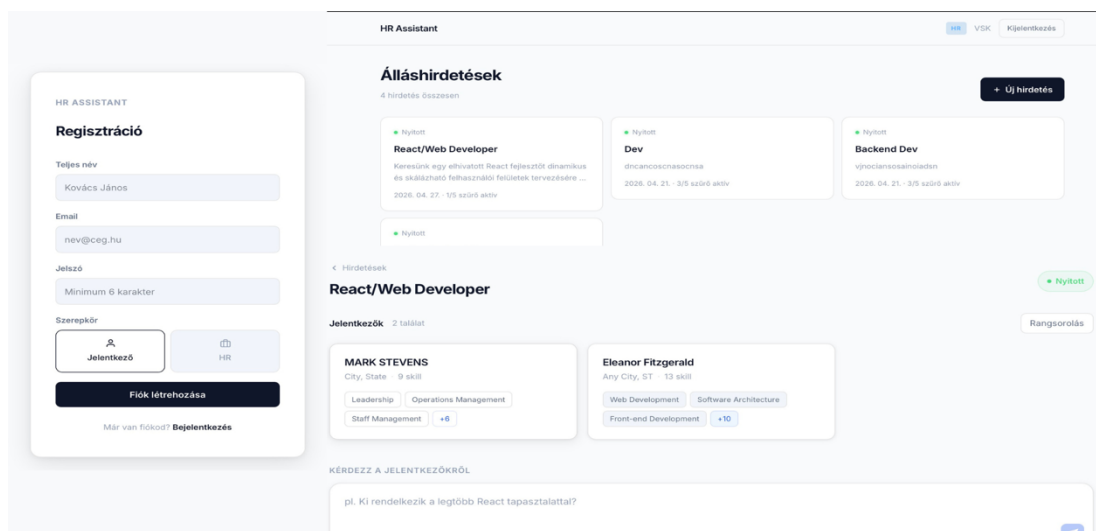
3. ábra Rangsorolás eredménye

4.5 Webalkalmazás – szerepkörök és felhasználói folyamatok

A Next.js frontend Supabase Auth-on keresztül kezeli a felhasználókat. A regisztráció során a felhasználó HR vagy Jelentkező szerepkört választ, amelyet a rendszer a felhasználói metaadatokban tárol, és az alkalmazás ennek alapján irányítja a megfelelő nézetbe.

HR szerepkörben a felhasználó álláshirdetéseket hozhat létre cím, leírás és egy anonimizálási konfiguráció megadásával. Ez utóbbi lehetővé teszi, hogy az öt anonimizáló agentet egyenként lehessen be- és kikapcsolni az adott pozícióhoz. Ez azt jelenti, hogy például egy olyan pozíciónál, ahol a HR tudatosan látni szeretné a neveket, a names agent kikapcsolható, miközben a többi agent fut. A HR dashboard az álláshirdetések listáját mutatja, lehetőséget nyújtva azok törlésére és a hirdetés nyitva/zárva állapotának váltására. Az egyes álláshirdetések részletező oldalán megtekinthetők a beérkezett, anonimizált önéletrajzok, és elindítható a rangsorolás.

Jelentkező szerepkörben a felhasználó a nyitott álláshirdetések listáját látja. Az egyes hirdetéseknel a Jelentkezés gomb megnyomásával PDF önéletrajzot tölthet fel. Feltöltés közben egy animált progress bar lépésről lépésre tájékoztatja az anonimizálás állásáról. Megjeleníti az éppen futó agentet (Nevek anonimizálása..., Nemi munkakörmegnevezések semlegesítése... stb.), így a felhasználó visszajelzést kap arról, hogy a feldolgozás zajlik és nem fagyott le a program. Ha az anonimizálás sikeres, a rendszer elmenti a jelölt adatait, és az önéletrajz anonimizált HTML formátumban megtekinthető a HR felületen. Ha a jelölt már korábban jelentkezett az adott álláshirdetésre, a rendszer ezt a Supabase adatbázis lekérdezésével érzékeli, és a Jelentkezés gomb helyett egy tájékoztató üzenetet jelenít meg, megakadályozva a duplikált jelentkezést. Hiba esetén, például, ha az anonimizáló pipeline háromszori próbálkozás után sem tudja eltávolítani az összes tiltott mintát, a backend 422-es hibakóddal tér vissza, amelyet a frontend egy egyértelmű hibaüzenetben jelenít meg a felhasználó számára, és a jelentkezés nem kerül mentésre.



4. ábra UI fontosabb részei

5. Tesztelés és mérési eredmények

5.1 Kísérletezési szakasz

A fejlesztés korai szakaszában, mielőtt az egyes komponensek bekerültek volna a tényleges backend kódbázisba, azokat egy elkülönített experiments mappában próbáltam ki. Ez a mappa a projekt gyökerében helyezkedik el, és két alkönyvtárból áll. Az agents almappa az öt anonimizáló agent önálló, egymástól független kísérletező verzióit tartalmazza, amelyeken keresztül az egyes promptok viselkedését elszigetelten lehetett tesztelni és finomítani, anélkül, hogy a változtatások azonnal hatással lettek volna az éles pipeline-ra. A langchain almappa a LangChain-alapú workflow korai változatait foglalja magában, amelyeket később LangGraph-ra migráltam. Ez a szétválasztás lehetővé tette, hogy az új megközelítéseket biztonságosan lehessen kipróbálni, és csak az elfogadott megoldások kerüljenek be a backend végleges kódjába.

5.2 Anonimizálási egységtesztek

Az anonimizáló pipeline helyességét tizenöt pytest egységteszttel ellenőriztem, amelyek a tests/test_anonymization.py fájlban találhatók. A tesztek négy csoportba szerveződnek,

amelyek együttesen az egyes agenteket önállóan, kikapcsolt állapotban, teljesen letiltott konfigurációban és kombinált módban is ellenőrzik.

Az első csoport öt tesztje az egyes agenteket önállóan futtatja, a többi letiltásával. Ezek ellenőrzik, hogy a megfelelő elemek valóban eltűnnek-e. A names agent hatására megjelenik-e a [CANDIDATE] helyőrző, a gendered_nouns agent után eltűnik-e a stewardess és a fireman kifejezés, a pronouns agent után eltűnnek-e a he és she névmások, a family_status agent után eltűnik-e a married szó, a life_events agent után pedig eltűnik-e a maternity leave kifejezés.

A második csoport szintén öt tesztből áll, és az ellenkezőjét ellenőrzi. Minden egyes agentet egyenként kikapcsol, miközben a többi fut, és megbizonyosodik arról, hogy a letiltott agent feladatkörébe tartozó elemek megmaradtak-e a szövegben. Ez fontos garancia arra, hogy az agenthatárok nem fedik át egymást, azaz az egyik agent nem végzi el a másik feladatát.

A harmadik csoport egyetlen tesztje azt ellenőrzi, hogy ha minden filter ki van kapcsolva, a szöveg a pipeline-on átmenve változatlan marad. Ez megerősíti, hogy a pipeline nem módosítja a tartalmat, ha arra nincs utasítás.

A negyedik csoport négy kombinációs tesztje komplex, valósághű önéletrajzon futtatja az agenteket különböző kombinációkban, és egyszerre több feltételt ellenőriz. A tesztelés során kiderült, hogy az eredeti promptok esetén az agenthatárok átfedtek. A pronouns agent nemi munkakörmegnevezéseket is semlegesített, a family_status agent pedig névmásokat cserélt [PERSONAL DATA]-ra. Ezeket a promptokat iteratívan finomítottam, kizárólag az adott agent hatáskörébe tartozó elemekre szűkítve a feladatot.

A tesztek futtatásához egy run_tests.sh szkriptet készítettem a projekt gyökerében, amellyel az összes teszt egyszerre, vagy egyenként, névvel megadva futtatható. Ha a lokális LLM nem elérhető, a tesztek automatikusan átlépésre kerülnek egy figyelmeztető üzenettel.

```
tests/test_anonymization.py::test_names_only PASSED
tests/test_anonymization.py::test_gendered_nouns_only PASSED
tests/test_anonymization.py::test_pronouns_only PASSED
tests/test_anonymization.py::test_family_status_only PASSED
tests/test_anonymization.py::test_life_events_only PASSED
tests/test_anonymization.py::test_names_skipped
```

5. ábra Tesztelés közbeni eredmények

5.3 RAGAS metrikák a RAG-rendszer értékelésére

A rangsorolást végző RAG-rendszer minőségét két RAGAS metrikával mértem, amelyek a tests/test_ragas_metrics.py fájlban találhatók. A metrikák a retrieval és a generálás minőségét numerikusan, nullától egyig terjedő skálán értékelik.

Az első metrika a Faithfulness, amely azt méri, hogy az LLM által generált rangsorolási indoklás valóban az önéletrajzban szereplő tényeken alapul-e, vagy az LLM az önéletrajzban nem szereplő információkat hallucinálja. A metrika az indoklás állításait egyenként ellenőrzi a visszakeresett CV-szöveg alapján, és az igazolt állítások arányával pontoz. Két teszteset készült. Egy valódi CV-adatokból merítő indoklásra, amelynél a várt pontszám legalább 0,5, és egy szándékosan kitalált tényeket tartalmazó indoklásra, amelynél a várt pontszám legfeljebb 0,5.

A második metrika a ContextRelevance, amely azt méri, hogy a vektoros kereséssel visszakeresett CV-szöveg mekkora hányada releváns az álláspozíció leírásához képest. A

metrika az LLM segítségével azonosítja a releváns mondatokat a szövegben, és azok arányával pontoz. Egy Python-fejlesztői pozícióhoz illeszkedő CV esetén a várt pontszám legalább 0,5, egy teljesen eltérő területről (séfként) érkező CV esetén legfeljebb 0,3.

6. Összefoglalás és továbbfejlesztési lehetőségek

Az önálló laboratóriumi munka során egy teljes körű, LLM-alapú HR asszisztens rendszert terveztem és valósítottam meg. A rendszer öt specializált agensből álló pipeline segítségével anonimizálja az önéletrajzokat, strukturált profilokat készít belőlük, majd RAG-technológiára épülő, kritériumalapú rangsorolást végez. Mindez egy felhasználóbarát webalkalmazáson keresztül érhető el, amely HR szakemberek és pályázók számára egyaránt egyértelmű és intuitív felületet nyújt.

A félév során a következő főbb eredményeket értem el. Megvalósítottam egy öt agenset tartalmazó, önellenőrző validációval és automatikus újrapróbálkozással rendelkező anonimizáló pipeline-t, felépítettem egy ChromaDB és Cohere multilinguális embeddingeket alkalmazó vektoros keresési infrastruktúrát, implementáltam egy kétlépéses, kritériumkinyerésen és rangsoroláson alapuló LLM-értékelő rendszert, elkészítettem egy teljes kliens-szerver webalkalmazást Supabase autentikációval és differenciált szerepkörkezeléssel, valamint tizenöt egységteszttel és RAGAS metrikákkal igazoltam a rendszer helyességét és a RAG-komponens minőségét.

A jövőbeli fejlesztési lehetőségek között szerepel a magyar nyelvű anonimizálás kibővítése, mivel a jelenlegi agent promptok elsősorban angol szövegekre vannak optimalizálva. Emellett a feladatkiírásban szereplő szintetikus tesztkörnyezet megvalósítása is tervezett. Egy generátormodul, amely kontrollált változókkal képes fiktív önéletrajzokat előállítani, lehetővé téve az alapmodell esetleges torzításának szisztematikus mérését. Hosszabb távon a rendszer skálázhatóságát párhuzamos feldolgozással és felhőalapú LLM integráció bevezetésével lehetne növelni.

7. Irodalomjegyzék

- [1] LangChain Documentation – LangGraph. <https://langchain-ai.github.io/langgraph/>
- [2] ChromaDB Documentation. <https://docs.trychroma.com/>
- [3] Cohere Embed Multilingual Model. <https://docs.cohere.com/docs/embed-multilingual>
- [4] RAGAS: Evaluation Framework for RAG Pipelines. <https://docs.ragas.io/>
- [5] PyMuPDF Documentation. <https://pymupdf.readthedocs.io/>
- [6] FastAPI Documentation. <https://fastapi.tiangolo.com/>
- [7] Supabase Documentation. <https://supabase.com/docs>
- [8] Next.js Documentation. <https://nextjs.org/docs>
- [9] Google Gemini API Documentation. <https://ai.google.dev/docs>
- [10] M. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", NeurIPS 2020.